

Dropped a Neural Net: Problem Parsing and Data Exploration

Source page: <https://huggingface.co/spaces/jane-street/droppedanet>

****Space metadata:**** likes=47, sdk=gradio, last_modified=2026-02-08T01:39:11.000Z

Puzzle Description (from README)

I dropped a neural net

Oh no! I dropped an extremely valuable trading model and it fell apart into linear layers! I need to rebuild it before anyone notices, but I can't remember how these pieces go together, or how it was trained. All I have left are the pieces of the model and some historical data. Can you help me figure out how to put it back together? Luckily I still have the source code of the layers that the neural network is made of. They look like this

```
class Block(nn.Module):
    def __init__(self, in_dim: int, hidden_dim: int):
        super().__init__()
        self.inp = nn.Linear(in_dim, hidden_dim)
        self.activation = nn.ReLU()
        self.out = nn.Linear(hidden_dim, in_dim)
    def forward(self, x):
        residual = x
        x = self.inp(x)
        x = self.activation(x)
        x = self.out(x)
        return residual + x

class LastLayer(nn.Module):
    def __init__(self, in_dim: int, out_dim: int):
        super().__init__()
        self.layer = nn.Linear(in_dim, out_dim)
    def forward(self, x):
        return self.layer(x)
```

The solution to this puzzle is a permutation. For each index from 0 to 96 (inclusive), I want you to give me the index of the piece that is applied in that position.

Hugging Face preview description

Enter a comma-separated list of 97 numbers (0-96) that represents the order of the model pieces. The app hashes your list and tells you whether it matches the hidden correct solution, giving you a ...

	zip_url
0	https://huggingface.co/spaces/jane-street/droppedaneuralnet/resolve/main/historical_data_and_pieces.zip

Preliminary Data Analysis

Analyzed file: `historical_data.csv`

Archive path: `data/droppedaneuralnet/historical_data_and_pieces.zip`

Rows: **10000** | Columns: **50** | Piece files: **97**

Archive contents

	filename	size_bytes
0	historical_data.csv	5579228
1	pieces/	0
2	pieces/piece_0.pth	20645
3	pieces/piece_17.pth	20453
4	pieces/piece_88.pth	20645
...	...	...
94	pieces/piece_61.pth	20645
95	pieces/piece_68.pth	20645
96	pieces/piece_46.pth	20453
97	pieces/piece_30.pth	20453
98	pieces/piece_84.pth	20645

99 rows × 2 columns

Customized exploration for this puzzle data

- 48 measurement features
- 97 model pieces in archive
- Piece shape mix is summarized in the inventory section below

Model quality summary from provided `pred` and `true`

	metric	value
0	MAE	0.171507
1	RMSE	0.326314
2	corr(pred,true)	0.940488

Top correlated measurements with `true`

	feature	corr_with_true
0	measurement_23	0.408260
1	measurement_40	0.320233
2	measurement_11	0.312609
3	measurement_31	0.310527
4	measurement_41	0.289374
5	measurement_2	0.277754
6	measurement_36	0.257540
7	measurement_8	0.250253
8	measurement_3	0.226445
9	measurement_42	0.222660

Working hypotheses from this dataset

	hypothesis	evidence
0	Provided <code>`pred`</code> approximates a strong baseline model output	corr(pred,true)=0.940, MAE/ σ (true)=0.179
1	Residual task signal is mostly linear and captured by measurements	Top feature corr with true reaches 0.408

	hypothesis	evidence
2	Error magnitude depends on target scale (heteroskedasticity check)	$\text{corr}(\text{error} , \text{true}) = -0.011$
3	Puzzle architecture likely 48 residual blocks + one final projection	Piece inventory contains 48 (96,48), 48 (48,96), and one (1,48) weight tensors.
4	Model fit remains imperfect, leaving recoverable puzzle signal	Estimated $R^2 = 0.884$ on <code>historical_data.csv</code>

Greedy residual reconstruction (seeded from final layer)

At each step, choose the remaining (96,48) expansion and (48,96) contraction pair that maximizes $\text{corr}(\text{model}(\text{measurements}), \text{pred})$ on a score sample.

	metric	value
0	final_layer_piece	85.000000
1	greedy_steps_completed	48.000000
2	best_full_corr_pred	0.651631

	step	expand_piece	contract_piece	score_corr_pred	full_corr_pred
0	1	42	55	0.274180	0.237308
1	2	61	25	0.380575	0.343229
2	3	95	82	0.451535	0.424773
3	4	18	34	0.509665	0.475810
4	5	39	54	0.553000	0.509071
...	...	...	...	...	...
15	16	0	17	0.711772	0.614419
16	17	59	90	0.720529	0.623872
17	18	73	20	0.725606	0.627843
18	19	16	38	0.731062	0.630548
19	20	84	70	0.735075	0.635178

20 rows × 5 columns

Schema and missingness checks

	column	dtype	missing
0	measurement_0	float64	0
1	measurement_1	float64	0

	column	dtype	missing
2	measurement_2	float64	0
3	measurement_3	float64	0
4	measurement_4	float64	0
...	...	...	...
45	measurement_45	float64	0
46	measurement_46	float64	0
47	measurement_47	float64	0
48	pred	float64	0
49	true	float64	0

50 rows × 3 columns

Piece inventory

	weight_shape	bias_shape	count
1	(48, 96)	(48,)	48
2	(96, 48)	(96,)	48
0	(1, 48)	(1,)	1

Piece index preview

	piece_index	weight_shape	bias_shape	file_size
0	0	(96, 48)	(96,)	20645
1	1	(96, 48)	(96,)	20645
2	2	(96, 48)	(96,)	20645
3	3	(96, 48)	(96,)	20645
4	4	(96, 48)	(96,)	20645
...	...	...	...	...
15	15	(96, 48)	(96,)	20645
16	16	(96, 48)	(96,)	20645
17	17	(48, 96)	(48,)	20453
18	18	(96, 48)	(96,)	20645
19	19	(48, 96)	(48,)	20453

20 rows × 4 columns

Raw data preview (48 measurement columns)

	measurement_0	measurement_1	measurement_2	measurement_3	measur
0	-0.871934	-0.762225	-0.757929	-0.721084	
1	-0.896439	-0.797072	-0.838245	-0.702223	-
2	1.585834	1.394803	0.797784	0.915132	
3	-0.814730	1.862772	0.659989	-0.388819	
4	-0.597139	0.370089	-0.906754	-0.331995	-
...	...	...	...	...	
15	-0.626517	-0.740302	-0.302393	1.934394	-
16	1.047458	0.152940	-0.926994	-0.659655	-
17	-0.744112	-0.669498	-0.831777	-0.801079	-
18	1.516115	1.973609	1.578767	1.188703	
19	-0.041643	0.006031	1.469229	1.602308	

20 rows × 50 columns